

LAB - 6 (Naiv Bays)

#Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

gnb = GaussianNB()
gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:-", accuracy)

print("Confusion Matrix", cm)
```

```
print(
    "Classification report=\n",
    classification_report(y_test, y_pred, target_names=iris.target_names)
)
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
```

```
plt.title("Confusion Matrix= Iris flower classification (GaussianNB)")
plt.xlabel("Predicted Species")
plt.ylabel("Actual species")
plt.show()
```

#IMDB Dataset.csv

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
df = pd.read_csv("IMDB Dataset.csv")

X = df['review']
y = df['sentiment'].map({'positive':1, 'negative':0})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("Training data shape=", X_train_tfidf.shape)
print("Training data shape=", X_test_tfidf.shape)

clf = MultinomialNB()

clf.fit(X_train_tfidf, y_train)

y_pred = clf.predict(X_test_tfidf)

cm = confusion_matrix(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:-", accuracy)

print("Confusion Matrix", cm)

plt.figure(figsize=(8,6))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Negative', 'Positive'],
    yticklabels=['Negative', 'Positive']
)
```

```
plt.title("Confusion Matrix= IMDB Movie Reviews")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
print(
    "Classification report=\n",
    classification_report(
        y_test,
        y_pred,
        target_names=['Negative', 'Positive']
    )
)
```

LAB – 7 (KNN Algorithm)

Creating won dataframe

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
```

```
import seaborn as sns
import matplotlib.pyplot as plt

X = np.array([[1],[2],[3],[4],[5],[6]])
y = np.array([0,0,0,1,1,1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("Train set:", X_train.flatten(), "Labels:", y_train)
print("Test set:", X_test.flatten(), "Labels:", y_test)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("\nPredicted labels:", y_pred)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

plt.figure(figsize=(6,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
new_hours = np.array([[2.5],[5.5]])
```

```
predictions = knn.predict(new_hours)
```

```
print("\nPredicted labels for {new hours} =", predictions)
```

spambase.data

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import confusion_matrix, classification_report
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = pd.read_csv("spambase.data", header=None)
```

```
X = data.drop(data.columns[-1], axis=1)
```

```
y = data[data.columns[-1]]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
```

```
)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train)
```

```
y_pred = knn.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

```
print(classification_report(y_test, y_pred))
```

LAB – 8 (SVM / SVR)

#SVM

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
```

```
df = pd.read_csv("Hotel Reservations.csv")
```

```
from sklearn.preprocessing import LabelEncoder
```

```
for col in df.select_dtypes(include=['object', 'string']).columns:
```

```
df[col] = LabelEncoder().fit_transform(df[col].astype(str))
```

```
print(df)
```

```
X = df.drop("booking_status", axis=1)
```

```
y = df["booking_status"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
scalar = StandardScaler()
```

```
X_train = scalar.fit_transform(X_train)
```

```
X_test = scalar.transform(X_test)
```

```
svm_clf = SVC(kernel='rbf', C=1, gamma='scale')
```

```
svm_clf.fit(X_train, y_train)
```

```
y_pred = svm_clf.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
import matplotlib.pyplot as plt
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
```

```
disp.plot()
```



```
plt.title("Confusion Matrix")  
plt.show()
```

#SVR

```
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler, LabelEncoder  
from sklearn.svm import SVR  
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score  
  
df_car = pd.read_csv("car_price_prediction.csv")  
  
for col in df_car.select_dtypes(include=['object']).columns:  
    df_car[col] = LabelEncoder().fit_transform(df_car[col])  
  
print(df_car)  
  
X = df_car.drop("Price", axis=1)  
y = df_car["Price"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
scalar = StandardScaler()  
  
X_train = scalar.fit_transform(X_train)  
X_test = scalar.transform(X_test)
```

```
svm_reg = SVR(kernel='rbf', C=100, gamma='scale')
```

```
svm_reg.fit(X_train, y_train)
```

```
y_pred = svm_reg.predict(X_test)
```

```
plt.scatter(y_test, y_pred, alpha=0.7, color='blue')
```

```
plt.xlabel("Actual Car Prices")
```

```
plt.ylabel("Predicted Car Prices")
```

```
plt.title("Car Price Prediction using SVM")
```

```
plt.plot(
```

```
    [y_test.min(), y_test.max()],
```

```
    [y_test.min(), y_test.max()],
```

```
    'r--'
```

```
)
```

```
plt.show()
```

```
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
print("MAE:", mean_absolute_error(y_test, y_pred))
```

```
print("R2:", r2_score(y_test, y_pred))
```

LAB – 9 (Artificial Neural Network)

CAR DETAILS FROM CAR DEKHO.csv

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neural_network import MLPRegressor
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("CAR DETAILS FROM CAR DEKHO.csv")
```

```
df.drop(['name'], axis=1, inplace=True)
```

```
df['age'] = 2026 - df['year']
```

```
df.drop(['year'], axis=1, inplace=True)
```

```
for col in ['fuel', 'seller_type', 'transmission', 'owner']:
    df[col] = LabelEncoder().fit_transform(df[col])
```

```
print(df)
```

```
X = df.drop(['selling_price'], axis=1)
y = df['selling_price']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
mlp = MLPRegressor(
```

```
hidden_layer_sizes=(64,32,16),
activation='relu',
solver='adam',
max_iter=2000,
random_state=42
)

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.2f}")
print(f"R2 Score: {r2:.4f}")

plt.scatter(y_test, y_pred)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted")

plt.show()
```

LAB – 10 (K mean Clustering)

Mall_Customers.csv

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import warnings

warnings.filterwarnings("ignore")

data = pd.read_csv("Mall_Customers.csv")

X = data.iloc[:, [3, 4]].values

wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss)

plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("WCSS")

plt.show()

kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42)
```

```
y_kmeans = kmeans.fit_predict(X)
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(  
    X[y_kmeans == 0, 0],  
    X[y_kmeans == 0, 1],  
    s=100,  
    c='red',  
    label='Cluster 1'  
)
```

```
plt.scatter(  
    X[y_kmeans == 1, 0],  
    X[y_kmeans == 1, 1],  
    s=100,  
    c='blue',  
    label='Cluster 2'  
)
```

```
plt.scatter(  
    X[y_kmeans == 2, 0],  
    X[y_kmeans == 2, 1],  
    s=100,  
    c='green',  
    label='Cluster 3'  
)
```

```
plt.scatter(  
    X[y_kmeans == 3, 0],  
    X[y_kmeans == 3, 1],
```

```
s=100,  
c='cyan',  
label='Cluster 4'  
)
```

```
plt.scatter(  
    X[y_kmeans == 4, 0],  
    X[y_kmeans == 4, 1],  
    s=100,  
    c='magenta',  
    label='Cluster 5'  
)
```

```
plt.scatter(  
    kmeans.cluster_centers_[:, 0],  
    kmeans.cluster_centers_[:, 1],  
    s=300,  
    c='yellow',  
    marker='*',  
    label='Centroids'  
)
```

```
plt.title("Clusters of Customers")  
plt.xlabel("Annual Income (k$)")  
plt.ylabel("Spending Score (1-100)")  
plt.legend()
```

```
plt.show()
```